Structure from Template Algorithm

Ksenia Krasikova[1*†], **Payal Priyadarshini**[1*†], **Philipp Harlos**[1*†]

[1]Department of Mathematics and Informatics, FU Berlin, Berlin, DE

***For correspondence:**
ksek00@zedat.fu-berlin.de (KK);
payap02@zedat.fu-berlin.de (PP);
harlop97@zedat.fu-berlin.de (QQ)

[†]These authors contributed equally to this work

Abstract

Goal of the project: Reconstructing 3D keypoints using the SfT algorithm shown in the lecture.
Main results: Accurate overall shape reconstruction but high RMSE values due to spatial misalignment.
Personal key learnings: Differences in SfT and NRSfM reconstruction algorithms and further Open3D implementation.
Estimated working hours: 8
Project evaluation: 3
Number of Words: 1416

Goal

This week's task is similar to last week's, in which we reconstruct a non-rigid 3D shape from a 2D projection. However, this week, we use a different approach called Shape-from-Template (SfT), which focuses on reconstructing a single target frame by projecting its sightlines onto a known 3D template shape. In contrast to last week, when we used Non-Rigid Structure-from-Motion (NRSfM), which iterated over all frames of a 2D projection to approximate the structure, SfT operates on a single frame at a time. However, this is only possible because we have a template 3D object on which we can cast the sightlines from our 2D key points, allowing us to have a reference to estimate the necessary distances.

The significant difference is that NRSfM iteratively estimates geodesic distances and depth depending on each other without prior 3D knowledge, whereas SfT relies on an existing 3D template to derive geodesic distances and uses that to constrain the optimization problem to find the depth. SfT, therefore, simplifies the problem by removing the need for iterative and dependent computations and the need for using all available frames instead of just using the template frame and the frame to be reconstructed. What both methods have in common, though, is that they both rely on camera intrinsics to compute sightlines, which are used to infer 3D keypoints from 2D keypoints by combining the sightlines with depth scaling.

Data

We worked with the Hulk dataset from last week. It consists of 10 frames with 2D keypoints and a ground truth reference with 3D keypoints. Since we work with the SfT approach, we need a template, and in our case, this will be Frame 8 of the HulkGT data, as this frame depicts a completely flat, undeformed poster. This planar structure ensures reliable geodesics. Additionally, SfT also requires camera intrinsic data, which is provided by the K.txt file. Before computing sightlines from the raw 2D key points, the shape of the key points was changed from (2xN) to (Nx2), but this was handled internally by the find_sightlines function.

Methods

Compute Sightlines & Reconstruct 3D Points

The pre-implemented SfT approach consists of two major parts: sightline creation and the optimization step that estimates depths along those sightlines using the template frame.

Identical to NRSfM, the sightline computation uses camera intrinsics to transform 2D key points into 3D direction vectors. Each 2D key point is first converted into a homogeneous coordinate by appending a 1, forming a 3D vector [x,y,1]. This vector is then multiplied by the inverse of the intrinsic matrix to map it into camera space. Finally, the resulting vector is normalized to unit length. The outcome of this is one sightline per key point, resulting in an array of shapes (N, 3, 1) where N is the number of key points. In order to parse the sightlines to the estimate_depths_from_sightlines function, squeeze() needs to be applied as the expected input is (Nx3).

Now, for the actual reconstruction, SfT follows a distinct approach. The 3D reconstruction is implemented in the estimate_depths_from_sightlines function, which takes three inputs: the previously computed sightlines, the template frame (3D keypoints), and the number of neighbors used to enforce inextensibility. This neighborhood-based constraint is similar to the k-nearest neighbors (kNN) used in NRSfM, but here it is applied to the template frame instead of the first frame of the input data.

The optimization begins by defining one depth variable per sightline, constrained to be non-negative, which intuitively makes sense. In contrast to NRSfM, we don't have to initialize a depth here; we only define constraints for the depth variable, which will then be applied in the optimization step. Next, the pairwise Euclidean distances between all 3D key points in the template are computed and stored in a matrix. For each point i, its k nearest neighbors in the template are identified. A constraint is then added to ensure that the distance between the reconstructed point i and each of its k nearest neighbors does not exceed the corresponding distance from the Euclidean distance matrix.

With these constraints defined, the optimization is called to maximize the sum of all depth values while satisfying the local inextensibility constraints enforced by the Euclidean distance matrix on the k nearest neighbors. The choice of k significantly influences the resulting shape: a lower k would allow more local deformation or more flexibility because only very few points would have to follow the template, while a larger k locks the shape reconstruction for larger areas to follow the template frame, allowing less local deformation compared to the template frame.

The final result of the optimization is the optimized depths under the constraints; these depths are then multiplied by the corresponding unit-length vectors to receive the 3D coordinates of the target object ultimately.

Evaluation

After optimization, the 3D reconstruction is plotted together with the ground truth 3D shape and colored by per-point RMSE. The evaluation of the reconstruction accuracy was pre-implemented using RMSE (Root Mean Squared Error). For the RMSE, the difference between each reconstructed point and its ground truth is computed, squared, and averaged across all points. The square root of this average gives the final RMSE. A lower RMSE indicates a better fit and higher accuracy.

Mesh Creation

In addition to the point cloud reconstruction plot, a triangular mesh was created using Open3D (***Zhou et al., 2018***). The reconstructed 3D points were turned into a point cloud, and surface normals were estimated and aligned. A Poisson reconstruction with depth 8 was used to generate the mesh. To make the mesh easier to see, it was colored based on Poisson density using a Viridis colormap.

Results & Discussion

For checking the accuracy of the 3D reconstruction, we did a comparison of the results to the known true 3D shape together with RMSE values (***Figure A1*** and ***Figure A2***) and a visual assessment of the reconstructed triangular mesh (***Figure A3***).

Generally, the reconstruction could capture most of the frames accurately, where the reconstructed shape reflects the ground truth. However, especially for frames one, four, and six, we get high RMSE values. These are roughly between 12 and 22 mm, resulting in a difference of 1.2-2.2 cm from the original shape. When looking into the point cloud reconstructions closely, it can be seen that although the general shapes of those frames could be reconstructed and are closely matching the reference, all three of those frames have a certain spatial offset between the two clouds, resulting in the high RMSE we observe.

Instead of using RMSE, which is very sensitive to misalignment, even though the reconstruction shape is correct, another metric called the Chamfer Distance (***Sim, 2020***) can be used. Instead of comparing corresponding point pairs directly, it computes the average distance from each point in one cloud to its nearest neighbor in the other cloud, and vice versa. This measures how closely two point clouds resemble each other, even if their point indices do not align. Additionally, a metric called the Hausdorff Distance (***Wikipedia contributors, 2024***) can be used. This metric considers the maximum distance of a point in one set to the closest point in the other set. Unlike Chamfer, which averages over all points, Hausdorff captures the worst-case mismatch between two point clouds. Both of these metrics can provide a more stable error estimate when the point clouds have similar shapes but are spatially offset.

Although the high RMSE can be attributed to spatial misalignment, it also can be seen that frames that depict the least amount of deformation compared to a flat, undeformed poster of frame eight tend to have lower RMSE. These range from roughly 2-6 mm, or 0.2-0.6 cm, where the lowest RMSE is in frame eight reconstruction. The cause of this can be that these shapes are easier to see clearly in the image and are also closer to the original template. This makes it easier to estimate the depths needed to reconstruct the correct 3D shape. Less folding means fewer hidden areas and fewer chances for mistakes during depth calculation.

Another minor observation was that while processing frame two, there is a numerical solver warning indicating that the solution may be inaccurate, which can increase frame two's reconstruction error.

The triangular mesh reconstruction generally captures the deformed poster's shape but shows roughness and uneven surface areas, especially in the central parts of the visualization. This rough appearance is likely caused by a sparse distribution of keypoints in those regions. A smoother and more realistic mesh could be achieved by either increasing the number of initial keypoints or by imputing new points in between existing ones.

Contributions

1. Triangular mesh with Open3D, error research and most part of error/accuracy reporting were done by Ksenia Krasikova.
2. Part of the report on accuracy over recontruction frames by Payal Priyadarshini.
3. Framewise 3D Reconstruction and most of the Goals and Methods part writing by Philipp Harlos.

References

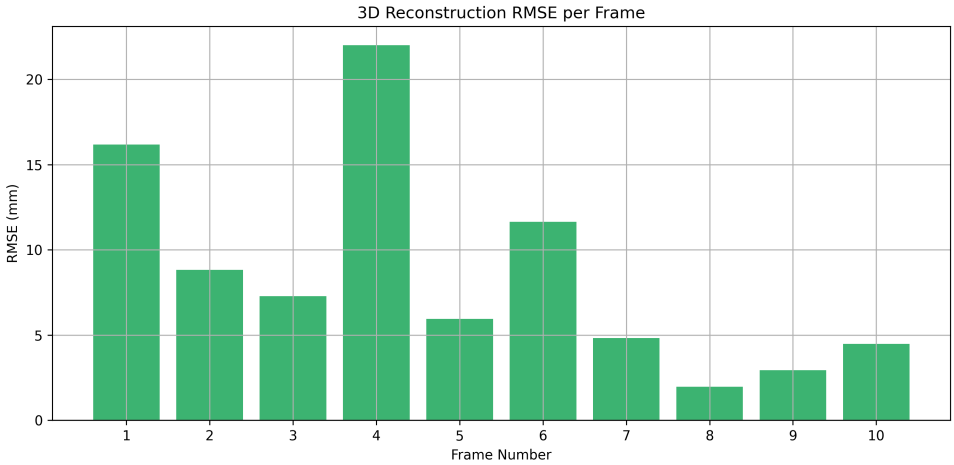
Sengupta A, Deformable 2D-3D Registration; 2025. Lecture slides, SOSE 2025 Geometric Computer Vision course. https://git.zib.de/asengupta/sose2025_geometric_cv.

Sim K, Chamfer Distance; 2020. Accessed: 2025-06-21. https://medium.com/@sim30217/chamfer-distance-4207955e8612.

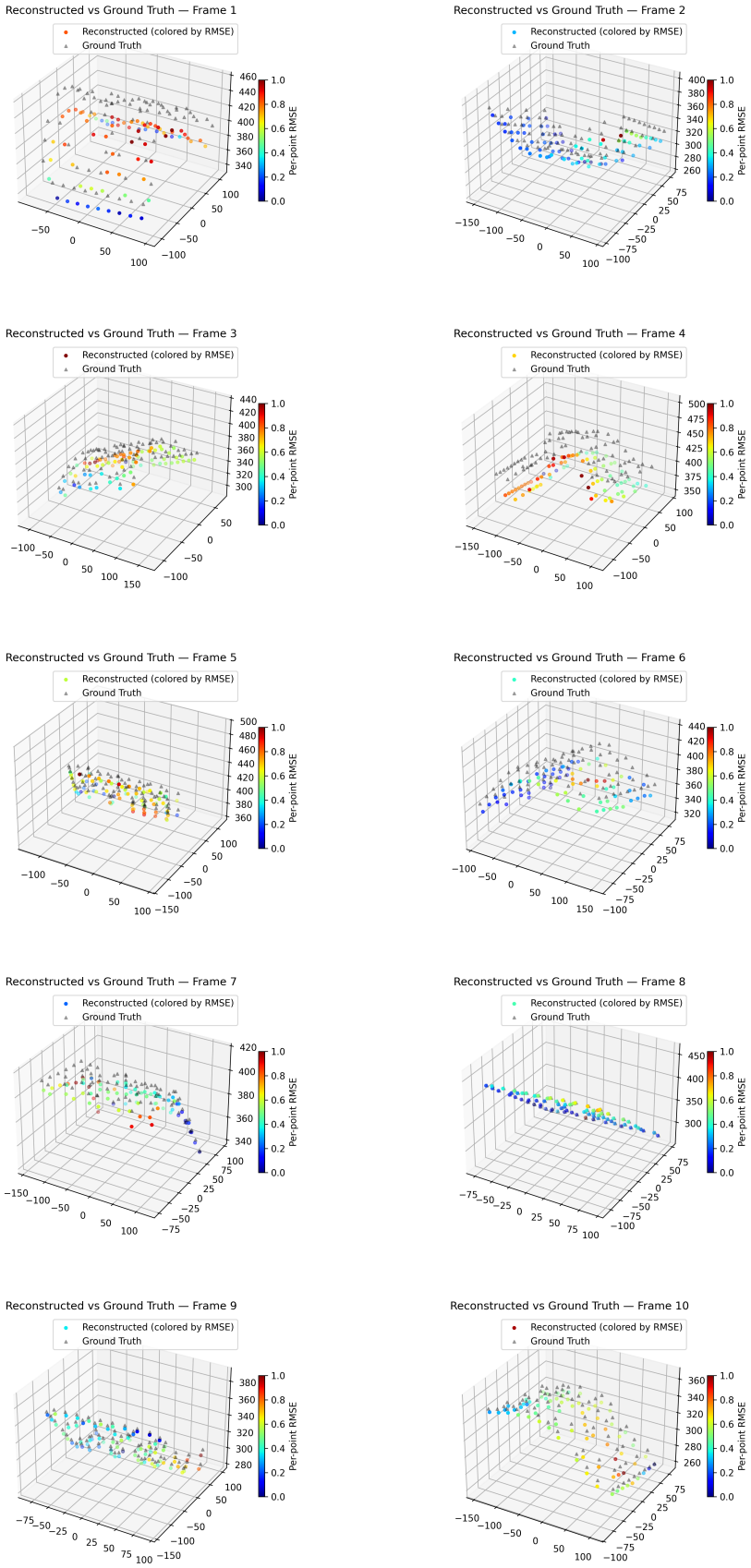
Wikipedia contributors, Hausdorff Distance; 2024. Accessed: 2025-06-21. https://en.wikipedia.org/wiki/Hausdorff_distance.

Zhou QY, Park J, Koltun V. Open3D: A Modern Library for 3D Data Processing. arXiv preprint arXiv:180109847. 2018; http://www.open3d.org.

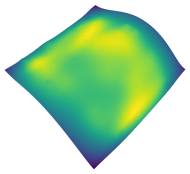
Supplementary Figures



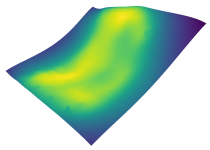
Appendix 0—figure A1. RMSE values for each reconstructed frame. Higher values indicate greater deviation from the ground truth 3D shape. One unit corresponds to one millimeter.



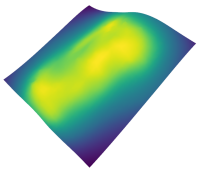
Appendix 0—figure A2. Visualization of the reconstructed 3D keypoints for frames 1–10. Each point is color-coded based on its RMSE (reconstruction error) relative to ground truth: **blue** indicates low error, while **red** indicates high error. The ground truth is shown in black for reference.



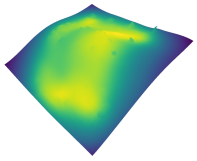
Frame 1



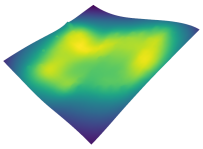
Frame 2



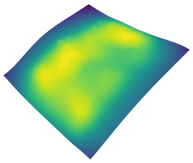
Frame 3



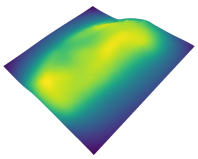
Frame 4



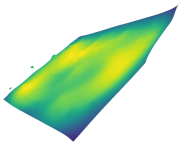
Frame 5



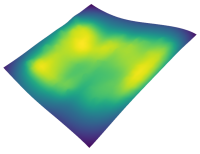
Frame 6



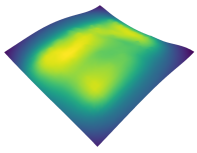
Frame 7



Frame 8



Frame 9



Frame 10

Appendix 0—figure A3. Triangular mesh reconstructions of the 3D keypoints for frames 1–10 using Open3D. Meshes were generated from the reconstructed point clouds via Poisson reconstruction and colored by vertex density.